IE437 · DATA-DRIVEN DECISION MAKING AND CONTROL — LECTURE 9

Optimal Control & Planning

The control-theoretic origin of sequential decision making

Jinkyoo Park · KAIST

Lecture 7 solved the dynamic optimum with operations research. *This is the other tradition.*

HANDOFF

The handoff — the second parent

Sequential decision making has two parents. Lecture 7 introduced the first. This lecture introduces the second.

Where we are — the other column of the map

STAGES	MODEL	AGENTS
dynamic ○ — ●	model-based ● — ○	single agent ● — ○

	LINEAGE A · OR / DYNAMIC PROGRAMMING	LINEAGE B · CONTROL THEORY
MODEL-BASED (ORIGIN)	**MDP & Dynamic Programming** LECTURE 7	**Optimal Control / Planning** LECTURE 9
DATA-DRIVEN (EXTENSION)	**Value-Based RL** LECTURE 8	**Policy-Based RL** LECTURE 10

The cube does not move: still **dynamic, model-based, single agent** — Lecture 7's cell exactly. What changes is the *tradition*. Lecture 7 solved sequential decision making the way **operations research** does; this lecture solves the **same problem** the way **control theory** does.

This is not a sequel to Lecture 8. It is **an independent root** — the second column of the grid, entered at the top.

Note the model axis: Lecture 8 threw the model away, and here it is handed back. Nothing has been undone — we are simply starting a *different* lineage at *its* model-based origin. Lecture 10 will then do to this lecture exactly what Lecture 8 did to Lecture 7.

Same problem, different dialect

The fixed point of the whole lecture, in one line:

Control theory and dynamic programming are **two dialects for one dynamic optimum.**

Where Lecture 7 had a stochastic transition over a finite set of states, control theory studies a system — usually continuous, often deterministic — driven by a control:

$$\dot{x}(t) = f\big(t, x(t), u(t)\big) \quad \text{or} \quad x_{k+1} = f_k(x_k, u_k), \qquad \min_u \int_0^T g\,dt \;+\; q\big(T, x(T)\big)$$

The goal is identical to Lecture 7's: not a *plan*, but a **feedback law** $u = \gamma(t, x)$ — the best control at every state and every time. The differences are of tradition, not of substance:

- finite action set $\to$ **continuous** control $u \in U \subseteq \mathbb{R}^m$;
- probabilistic transition $T(s, a, s') \to$ **differential** dynamics f;
- sum $\to$ integral; reward and $\max \to$ cost and $\min$; a backup $\to$ a **PDE**.

One wall Lecture 8 left, and where it actually lands

the continuous-argmax problem — a meeting point, not the premise

Lecture 8 ended at a wall: with a continuum of actions, $\max_{a'} Q(s', a')$ is itself an intractable search, so Q-learning's update cannot even be written down.

That wall is not this lecture's starting point — we begin from Lecture 7, not Lecture 8. But it *is* one of the seams where the two lineages meet, and it is worth naming now because it recurs three times today:

WHERE	THE SAME INNER PROBLEM	WHAT HAPPENS TO IT
Act 1 · the DP backup	$\min_{u \in U}\{g + V(f(x,u))\}$ over a continuum	it becomes a nested Lecture-1 problem
Act 3 · LQR	the identical $\min$, with g quadratic and f linear	**solved in closed form**, one line of algebra
Act 4 · Pontryagin	$\min_{u \in U} H$ at each instant	reduced to a *static* minimisation

Control theory has lived with the continuous $\arg\min$ since 1956, and its three answers are today's acts. Lecture 10's answer — *learn a network that outputs the minimiser* — is the fourth, and it is a descendant of the third.

The translation table — Lecture 7 in continuous time

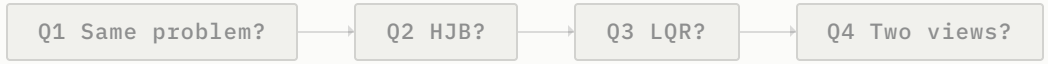
Every object of dynamic programming has a control-theoretic twin. Keep this in sight all lecture.

LECTURE 7 (OR / DYNAMIC PROGRAMMING)	LECTURE 9 (CONTROL)	THE SHIFT
transition $T(s, a, s')$	dynamics $\dot{x} = f(t, x, u)$	probabilistic $\to$ differential
reward R**, maximise**	cost g, minimise	a sign, and a culture
value $V(s)$**,** $Q(s, a)$	cost-to-go $V(t, x)$	state $\to$ state *and* time
policy $\pi(s)$	feedback law $u = \gamma(t, x)$, $u = -Kx$	a rule, now continuous-valued
Bellman optimality (sum, $\max$**)**	HJB equation (PDE, $\min$)	a backup $\to$ a PDE
value iteration	the Riccati recursion	**the same sweep, on a matrix**
policy iteration / LP	Riccati / Pontryagin	a solver, not a sampler

Read it as one claim: **HJB is the Bellman optimality equation made infinitesimal.** Lecture 7's intuition transfers wholesale; only the calculus changes. And keep the last two rows in view — that is the seam where, in Lecture 10, samples will replace the solver.

The roadmap — four questions

One question per Act. This strip returns at every transition — watch the highlight move.



- **Q1 — Is this really the same problem?** Discrete-time optimal control *is* Lecture 7's dynamic programming, made deterministic and continuous-valued.
- **Q2 — What is Bellman in continuous time?** The Hamilton–Jacobi–Bellman PDE: sufficient, global, and almost never solvable. *(Bellman, 1957)*
- **Q3 — When *can* we solve it?** The one closed form in the whole course: LQR and the Riccati equation. *(Kalman, 1960)*
- **Q4 — Is there another way in?** Pontryagin's minimum principle — the trajectory view, and the costate. *(Pontryagin et al., 1956)*

ACT 1

It is the same problem

Q1. Control theory writes different symbols. Is it solving anything different?

Discrete-time optimal control — the problem, stated

Q1 Same problem? → Q2 HJB? → Q3 LQR? → Q4 Two views?

Choose a control to minimise an accumulated cost subject to deterministic dynamics:

$$L(u) = \sum_{k=0}^{K-1} g_k(x_k, u_k) + g_K(x_K), \qquad x_{k+1} = f_k(x_k, u_k), \quad u_k \in U_k$$

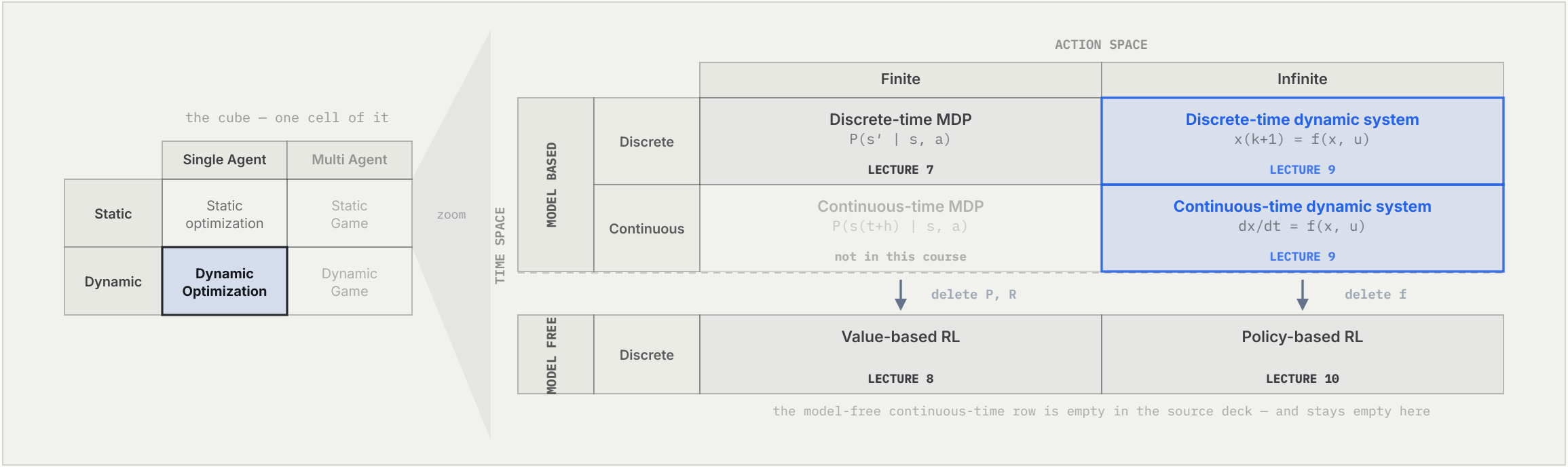
Read the third clause carefully, because it is the whole lecture in one symbol:

$$u_k = \gamma_k(x_k), \qquad \gamma_k \text{ a } \textit{permissible control strategy} \text{ at stage } k$$

What we seek is not a sequence of numbers $u_0, \dots, u_{K-1}$ but a **rule** γ_k from state to control. Control theory insists on this from its very first slide, for the same reason Lecture 7 wanted a policy rather than a plan: a rule is still valid from wherever the system actually lands.

The same 2×2, one level down

Lecture 7 set this out as two tables — here it is drawn whole



The course cube puts Lectures 7 and 9 in **one cell**: single agent, dynamic, model-based. Zoom into that cell and two finer coordinates appear — how large the action set is, and whether time ticks or flows. **Lineage A is the finite column; lineage B is the infinite column**, and deleting the model is a single vertical arrow that acts on both at once.

Lecture 7 read this grid across the *columns*, to say what separates it from today. Read it down the *rows* instead and it says something Lecture 7 could not yet: the model axis is one move, made twice — and the empty bottom row is not an oversight but the honest edge of the field.

Dynamic programming, in control's notation

Define the cost-to-go from *any* state x at *any* time k:

$$V(k,x) = \min_{\gamma_k,\dots,\gamma_K} \sum_{i=k}^{K} g_i(x_i,u_i), \qquad u_i = \gamma_i(x_i) \in U_i, \quad x_k = x$$

BELLMAN'S PRINCIPLE OF OPTIMALITY — quoted as the source deck quotes it

An optimal policy has the property that whatever the initial state and initial decision are, **the remaining decisions must constitute an optimal policy** with regard to the state resulting from the first decision.

Applied directly, it gives the recursion — and it is Lecture 7's, letter for letter:

$$V(k,x) = \min_{u_k \in U_k} \Big\{ g_k(x,u_k) \; + \; V\big(k+1, \; f_k(x,u_k)\big) \Big\}$$

Each u_k^* is read off as the argument attaining the right-hand side — what the source deck calls a **single-shot optimisation**. That phrase is worth keeping: a problem that unfolds over time has been reduced to one static optimisation performed once per instant. We meet the move again in Act 2 and again in Act 4.

Where the expectation went

LECTURE 7 — THE BELLMAN BACKUP

$$V(s) = \max_a \sum_{s'} T(s,a,s')\left[R + \gamma V(s')\right]$$

- a **distribution** over successors,
- a **finite** set of actions to enumerate,
- **reward**, and $\max$.

LECTURE 9 — THE DP RECURSION

$$V(k,x) = \min_u \left\{ g_k(x,u) + V\big(k{+}1,\, f_k(x,u)\big) \right\}$$

- **one** deterministic successor,
- a **continuum** of controls,
- **cost**, and $\min$.

Two of those three differences are cosmetic. The expectation $\sum_{s'} P[\cdots]$ collapses to a single point because the dynamics are deterministic — put the noise back and it returns unchanged. Cost versus reward is a sign.

Solve backward in time; optimal sub-plans compose into an optimal plan. The logic is identical.

The one real difference — the $\arg\min$ runs over a continuum

In Lecture 7 the inner $\max_a$ was a *loop*: try each of the finitely many actions, keep the best. Here U_k is a continuum, so the inner minimisation

$$\min_{u \in U_k} \underbrace{g_k(x,u) + V\big(k+1, f_k(x,u)\big)}_{\text{a function of } u}$$

is not a loop at all. It is an optimisation problem — **Lecture 1's template, nested inside every backup**, once per state and once per time step.

So the continuous action space is not a nuisance detail; it is the structural fact that makes this a different tradition. Everything that follows is a way of coping with it: make the inner problem *smooth* so calculus applies (Act 2), make it *quadratic* so it has a closed form (Act 3), or characterise its solution *along one trajectory* rather than everywhere (Act 4).

Same problem, same principle — but the backup now contains **an optimisation, not a loop.**

ACT 2

Bellman made infinitesimal

Q2. Let the time step shrink to nothing. What does the Bellman equation become?

Continuous time — the problem, and the cost-to-go

Time no longer ticks; it flows.

$$L(u) = \int_0^T g\big(t, x(t), u(t)\big)\, dt \;+\; q\big(T, x(T)\big), \qquad \dot{x}(t) = f\big(t, x(t), u(t)\big), \quad x(0) = x_0$$

with $u(t) = \gamma\big(t, x(t)\big) \in U$ and $\gamma \in \Gamma$, the class of **admissible feedback strategies**.

The cost-to-go is defined exactly as before, over the remaining interval:

$$V(t,x) = \min_{u(s),\, t \le s \le T} \Big[\int_t^T g\big(s, x(s), u(s)\big)\, ds + q\big(T, x(T)\big) \Big], \qquad V(T,x) = q(T,x)$$

The boundary condition is the terminal cost. In Lecture 7 the recursion was seeded at $V(K,\cdot) = g_K$; here it is seeded at $t = T$. Same seed, same backward direction.

Shrink the step — and a backup becomes a PDE

Apply the principle of optimality over a short interval δ, then let $\delta \to 0$:

$$V(t,x) \;=\; \min_{u \in U} \Big\{\, g(t,x,u)\,\delta \;+\; V\big(t+\delta,\; x + f(t,x,u)\,\delta\big) \,\Big\}$$

Taylor-expand the second term, cancel $V(t,x)$ from both sides, divide by δ and take the limit. Assuming V is continuously differentiable, what survives is the **Hamilton–Jacobi–Bellman equation**:

$$-\frac{\partial V(t,x)}{\partial t} \;=\; \min_{u \in U} \Big\{ \frac{\partial V(t,x)}{\partial x}\, f(t,x,u) \;+\; g(t,x,u) \Big\}, \qquad V(T,x) = q(T,x)$$

This *is* the Bellman optimality equation. It has become a nonlinear PDE for V.

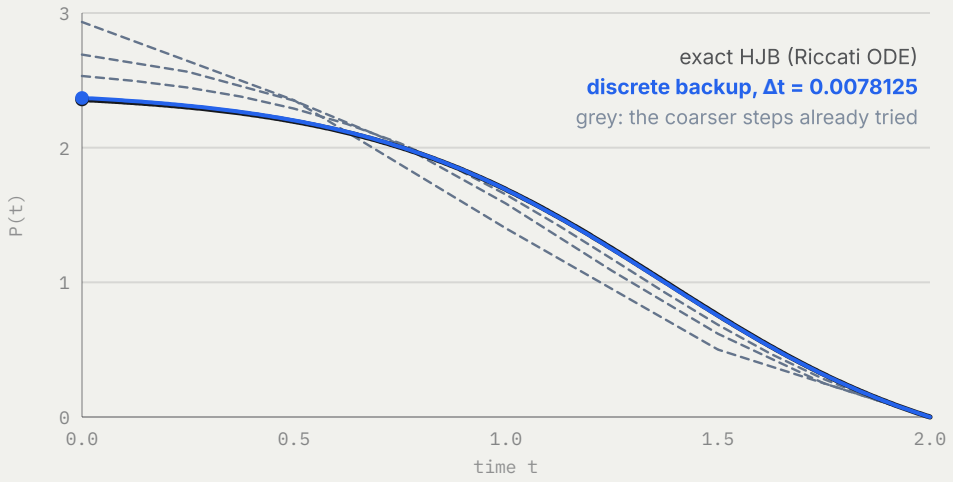
The full three-line derivation is Backup 1. Compare it with Lecture 7's: identical, with $\delta \to 0$ where Lecture 7 had a one-step sum.

The claim, checked

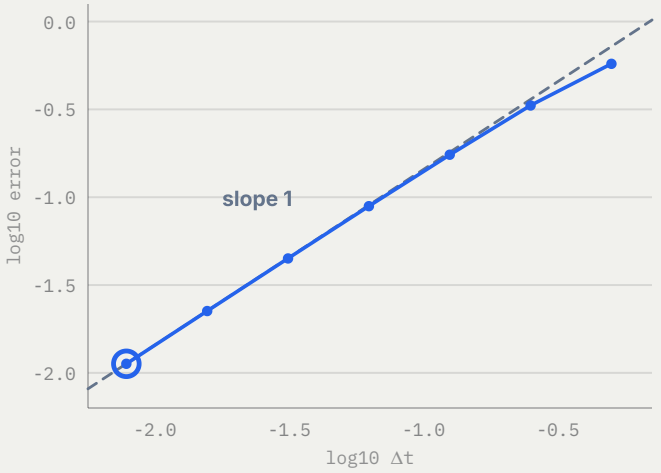
shrink Δt on a discrete backup and watch it land on the PDE

SHRINK THE STEP AND THE BACKUP BECOMES THE PDE STEP **0.0078125**

COST-TO-GO COEFFICIENT P(T) — EXACT HJB VS THE DISCRETE BACKUP



ERROR AT T = 0, BOTH AXES LOG10



AT T = 0

exact **2.357772**

backup **2.369035**

error **1.126e-2**

halving Δt divided the error by **1.996**

observed order 0.997 — first, as one Euler step should be

A scalar problem whose HJB solution is known exactly: $\dot{x} = x + u$, cost $\int_0^2 (x^2 + u^2)\, dt$, so $V(t,x) = P(t)x^2$ with P solving a Riccati ODE. The *discrete* backup of Act 1 is run on the Euler-discretised system at step Δt and its P_k plotted against the exact curve. Halve Δt and **the error halves** — first order, exactly as an Euler step should be.

What HJB buys — a certificate, not just a formula

SUFFICIENT, AND GLOBAL

Find *any* continuously differentiable V solving the PDE and you have proved optimality — and the optimal control falls out of a **static, pointwise** minimisation:

$$u^*(t) = \arg\min_{u \in U} \Big\{ \partial_x V \, f(t,x,u) + g(t,x,u) \Big\}$$

FEEDBACK, EVERYWHERE

V is a field over the *whole* state space, so the rule it generates is credible from **any state the system actually reaches** — not only the one you predicted.

And it extends to stochastic dynamics with one extra term, $\tfrac{1}{2}\mathrm{tr}\big(\sigma\sigma^\top \partial_{xx} V\big)$.

"Sufficient" is a strong word and it is earned: a solution of the PDE is a *certificate* that every other admissible trajectory costs at least as much. The four-line verification argument is Backup 2, and it is the only place in this lecture where something is proved optimal rather than merely derived.

And what it costs

HJB is a **nonlinear PDE over the entire state space.**

Three prices, and each is a limitation we will spend the rest of the course working around:

- **It needs the model.** f and g appear explicitly inside the minimisation. Delete them and there is no equation left to solve — which is Lecture 10.
- **It assumes V is differentiable.** Often it is not; the honest fix is a weaker notion of solution (viscosity solutions), which buys existence at the cost of the clean formula. *(Crandall & Lions, 1983)*
- **It does not survive dimension.** A value *field* over $x \in \mathbb{R}^n$ is exponentially large — the same curse Lecture 7's table faced, now with the table replaced by a continuum.

So HJB is the **characterisation**, rarely the computation. It is exactly solvable in essentially one case — and it happens to be the case every control engineer knows cold.

ACT 3

The one closed form

Q3. Linear dynamics, quadratic cost. The only sequential decision problem in this course with an exact answer you can write down.

LQR — the harmonic oscillator of control

Q1 Same problem? → Q2 HJB? → **Q3 LQR?** → Q4 Two views?

$$\dot{x} = Ax + Bu, \qquad J = \int_0^T \left(x^\top Q x + u^\top R u\right) dt + x(T)^\top Q_f\, x(T), \qquad Q \succeq 0,\ R \succ 0$$

Try $V(t,x) = x^\top P_t\, x$. Then $\partial_x V = 2P_t x$ and $\partial_t V = x^\top \dot{P}_t x$, and the HJB equation reads

$$-x^\top \dot{P}_t x \;=\; \min_u \Big\{\, 2x^\top P_t\big(Ax + Bu\big) + x^\top Q x + u^\top R u \,\Big\}$$

The inner minimisation — the continuum that Act 1 warned about — is now a **convex quadratic in u**. Set the gradient to zero:

$$2B^\top P_t x + 2Ru = 0 \quad\Longrightarrow\quad u^*(t) = \underbrace{-R^{-1}B^\top P_t}_{-K_t}\, x(t)$$

The wall of the continuous $\arg\min$ falls to **one line of linear algebra.**

The Riccati equation, and a gain that stops moving

Substitute u^* back and match $x^\top(\cdot)x$ for all x. What is left is an ODE in the matrix P_t alone:

$$-\dot{P}_t = A^\top P_t + P_t A - P_t B R^{-1} B^\top P_t + Q, \qquad P_T = Q_f$$

the **Riccati differential equation**, integrated backwards in time — a backward sweep, exactly like Lecture 7's.

Let the horizon go to infinity. The problem becomes *shift invariant* — the time-to-go is always ∞ — so V cannot depend on t, and $\dot{P} \to 0$. What remains is algebra:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0 \qquad \text{(the algebraic Riccati equation)}$$

And so the optimal controller is a matrix: $u^* = -Kx,\ K = R^{-1}B^\top P.$

The discrete-time problem $x_{t+1} = Ax_t + Bu_t$, $J = \sum(x^\top Qx + u^\top Ru)$ runs the identical argument and lands on $P = Q + A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA$ with $K = (R + B^\top PB)^{-1}B^\top PA$. Backup 3 carries both derivations in full.

The Riccati recursion is value iteration

The single sharpest bridge between Lecture 7 and Lecture 9, and it comes free with the discrete-time form.

$$P_{k+1} = Q + A^\top P_k A - A^\top P_k B\left(R + B^\top P_k B\right)^{-1} B^\top P_k A, \qquad P_1 = Q$$

LECTURE 7	LECTURE 9
$V_{k+1} \leftarrow \max_a \mathbb{E}[R + \gamma V_k]$	$P_{k+1} \leftarrow$ the Riccati map
V_k **is a table, one entry per state**	P_k is an $n \times n$ **matrix**
converges to the unique fixed point V^*	converges to the **unique PSD solution** of the ARE
a sweep touches every state	a sweep is $O(n^3)$ — over a *continuum* of states

Value iteration, with the value function carried in closed form instead of enumerated. The uniqueness of the positive-semidefinite P plays the role of the uniqueness of the Bellman fixed point, and — the source deck's own remark — **infinite-horizon LQR is just steady-state finite-horizon LQR.** One backward sweep, run to convergence.

Two hazards the tidy derivation hides

HAZARD 1 — THE COST CAN BE INFINITE

$$x_{t+1} = 2x_t + 0 \cdot u_t, \qquad x_0 = 1$$

An unstable mode the input cannot touch: $J = \infty$ for *every* input sequence.

The condition that rules it out is **controllability of** (A, B) — then some input drives x to zero in n steps and holds it, so $\min_u J < \infty$ from every x_0.

HAZARD 2 — THE WEIGHTS ARE NOT FREE

$Q \succeq 0$ makes the cost a cost. But $R \succ 0$ — *strictly* — is what makes the inner minimisation of Act 1 well-posed: with R singular, the quadratic in u has no minimum and R^{-1} does not exist.

Free control is not a limiting case of cheap control. It is a different problem.

Controllability is the control lineage's counterpart of Lecture 8's "every state–action pair visited infinitely often": a reachability condition, assumed quietly, without which the theorem is false rather than merely slow.

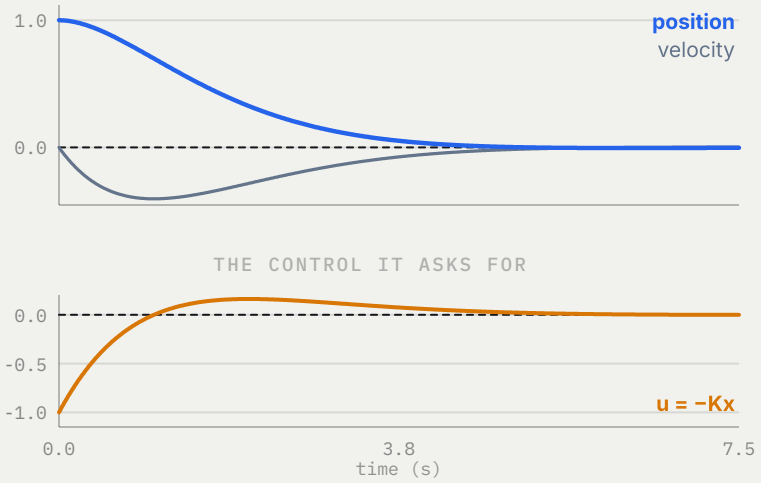
LQR, run

a double integrator — position and velocity as the state, force as the input

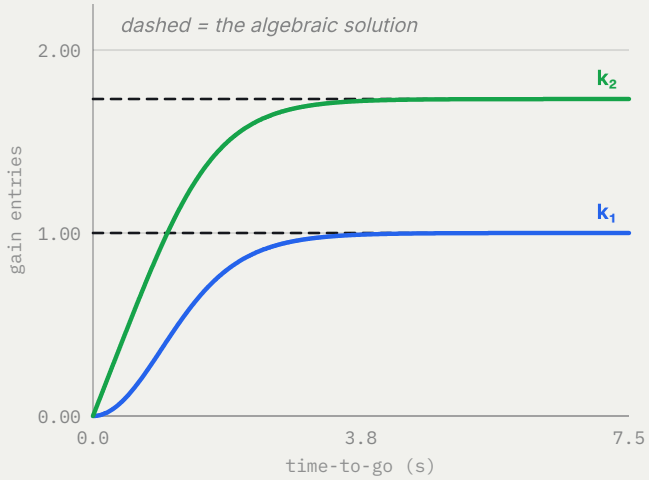
A MASS YOU CAN PUSH — THE DIALS *ARE* THE OBJECTIVE

STATE COST Q **1** CONTROL COST R **1**

CLOSED LOOP FROM POSITION 1, AT REST



THE GAIN STOPS MOVING — RICCATI, BACKWARDS



THE OPTIMAL CONTROLLER

u = −[1.000, 1.732] x

closed form

$k_1 = \sqrt{(Q/R)} = 1.000$

$k_2 = \sqrt{((2\sqrt{(QR)}+1)/R)} = 1.732$

✓ the sweep matches it to 2e-5 by the end of the window, 6e-15 given four times as long

poles **-0.87 ± 0.50i**

optimal cost **1.732** = $x_0^T P x_0$

peak |u| **1.00**

$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ — a mass you can push. Turn the dials and read the gain. The Riccati integration on the right **flattens onto the algebraic solution**, which is drawn as a dashed line from the closed form $k_1 = \sqrt{q/r}$, $k_2 = \sqrt{(2\sqrt{qr}+1)/r}$. Cheap control ($R$ small) buys speed with a violent input; expensive control is gentle and slow. There is no tuning here — the dials *are* the objective.

Why LQR is the cornerstone

- **Globally optimal, and linear.** For the LQ problem there is no iteration, no approximation and no local minimum. The answer is a matrix.
- **The local model of everything.** Linearise any smooth problem about a trajectory, take a quadratic approximation of the cost, solve the resulting LQR, re-linearise, repeat: that is **iLQR / DDP**, and it is the backbone of trajectory optimisation and of model-based planning in Lecture 11. *(Jacobson & Mayne, 1970)*
- **The target the data-driven world reaches for.** In Lecture 10, DDPG's learned actor $\mu_\theta(s)$ occupies exactly the place of this gain K — for a system whose A and B are **unknown**.

Hold $u = -Kx$ in view: it is the closed form that policy gradient **learns to approximate blind.**

ACT 4

The other view

Q4. HJB carries a value field over every state. Is there a way in that follows only one trajectory?

Adjoin the dynamics — the Hamiltonian and the costate

HJB is the *Eulerian* view: stand still and watch a field $V(t, x)$ over all states. There is a *Lagrangian* view: ride the optimal trajectory and enforce the dynamics as a constraint along it.

This is Lecture 1's Lagrangian move, applied once per instant. Adjoin $\dot{x} = f$ with a multiplier $\lambda(t)$ — the **costate** — and collect the terms into the **Hamiltonian**:

$$H(t, x, u, \lambda) \;=\; g(t, x, u) \;+\; \lambda^{\top} f(t, x, u)$$

One multiplier per instant, and a whole function $\lambda(\cdot)$ instead of a vector. The constraint set of Lecture 1 has become the dynamics themselves.

The necessary conditions

Along an optimal trajectory, four conditions hold together:

$$\dot{x}^* = \frac{\partial H}{\partial \lambda} = f, \qquad \dot{\lambda}^* = -\frac{\partial H}{\partial x}, \qquad \lambda(T) = \frac{\partial q}{\partial x}\Big|_{x^*(T)}, \qquad u^* = \arg\min_{u \in U} H(t, x^*, u, \lambda^*)$$

Read the shape of it. The state runs **forward** from $x(0) = x_0$; the costate runs **backward** from $\lambda(T)$; and they are coupled. This is a **two-point boundary-value problem**, not an initial-value problem — you cannot simply integrate.

And the last condition is the familiar move again: dynamic optimisation reduced to **a static minimisation of H, freshly at each instant.**

What the costate actually is

THE COSTATE IS THE VALUE GRADIENT, SEEN FROM THE TRAJECTORY

$$\lambda(t) \;=\; \frac{\partial V}{\partial x}\big(t, x^*(t)\big)$$

HJB carries $\partial_x V$ over *every* state; Pontryagin carries its restriction to the **one trajectory you are on**. That is precisely the saving, and precisely the loss.

Check it on the scalar LQ problem of Act 3, where $V(t,x) = P(t)x^2$ and so $\lambda = 2P(t)x$. Substituting into $\dot\lambda = -\partial_x H$ reproduces the Riccati equation exactly — the same optimum, reached from the other side.

Economically, λ is a shadow price: the marginal cost of being nudged in state x at time t. It is Lecture 1's multiplier with a time index — and it is what a policy gradient will later estimate by sampling instead of solving.

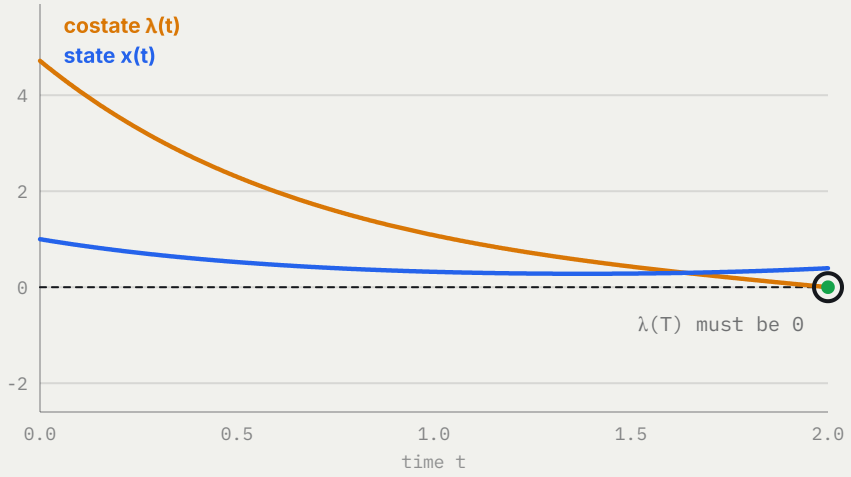
Shooting — two views, one optimum

guess the costate, integrate, and see whether it lands

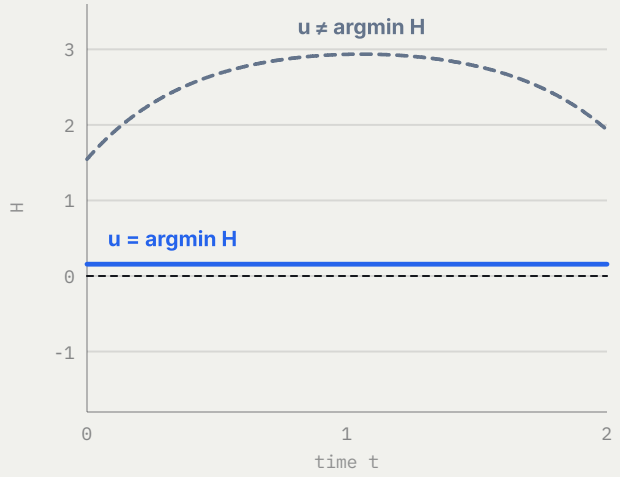
GUESS THE COSTATE, INTEGRATE, SEE WHETHER IT LANDS

COSTATE AT T = 0 **4.7155**

STATE FORWARD, COSTATE TO ITS TERMINAL CONDITION



THE HAMILTONIAN ALONG THE TRAJECTORY



THE TWO-POINT PROBLEM

λ(0) **4.715543**

λ(T) **0.000000** target 0

cost J **2.3649**

✓ **it lands.** And HJB, solved independently, says λ*(0) = 2P(0)x_0 = **4.715543** — agreeing to **13** digits.

H varies by **5e-15** obeying the principle, **1.39** not obeying it.

The same problem the HJB widget solved: $\dot{x} = x + u$, $\int_0^2 (x^2 + u^2) dt$. Guess $\lambda(0)$, integrate the coupled equations forward, and check whether $\lambda(2) = 0$ as the transversality condition demands. Exactly one guess lands — and it is $\lambda^*(0) = 2P(0)x_0$, the value gradient from the *other* method. The dashed trace is a control that does *not* minimise H pointwise — and its Hamiltonian, conserved to fourteen digits when the principle is obeyed, drifts.

HJB versus Pontryagin — two solvers, one optimum

	HJB *(dynamic programming)*	Pontryagin *(minimum principle)*
view	Eulerian — a value **field** $V(t, x)$	Lagrangian — one **trajectory** $x^*(t)$
logic	**sufficient**, over all (t, x)	**necessary**, along the optimum
output	a feedback law $\gamma(t, x)$	an open-loop $u^*(t)$, from a costate ODE
mathematics	a nonlinear **PDE**	a two-point boundary-value **ODE**
stochastic	extends naturally	does not, in general
cost	the curse of dimensionality	one trajectory, but only local

Feedback beats an open-loop plan for the reason Lecture 7 already gave: $\gamma(t, x)$ is valid from *any* state you actually reach, while $u^*(t)$ is valid only along the trajectory you predicted. But Pontryagin scales where HJB cannot, which is why every practical trajectory optimiser is built on it. Its machinery returns in Lecture 10 — REINFORCE is a trajectory view — and in Lecture 11, where a learned policy imitates an iLQR teacher.

CLOSING

Closing

Both model-based origins are now on the table. Each one leaned entirely on knowing f.

Where we are — both parents, both exact

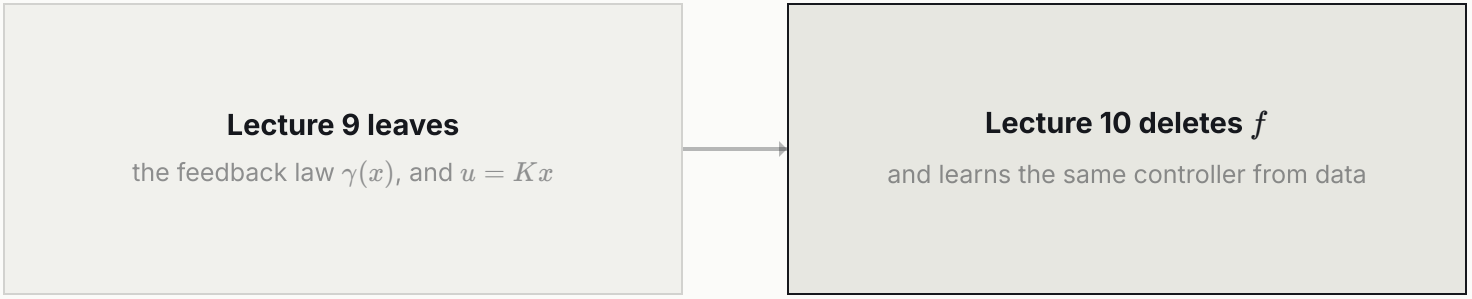
	LINEAGE A · OR / DYNAMIC PROGRAMMING	LINEAGE B · CONTROL THEORY
MODEL-BASED (ORIGIN)	**MDP & Dynamic Programming** LECTURE 7	**Optimal Control / Planning** LECTURE 9
DATA-DRIVEN (EXTENSION)	**Value-Based RL** LECTURE 8	**Policy-Based RL** LECTURE 10

Three exact answers to one question, and no approximation anywhere in them:

	WHAT IT GIVES	WHAT IT NEEDS
HJB	the characterisation — V over all (t, x), and feedback	f, g, and a differentiable V
LQR	the one closed form — $u = -Kx$ from the Riccati equation	A, B, Q, R
Pontryagin	one optimal trajectory, and the costate	f, g, and $\partial_x f$

Each is exact. **Each needed the dynamics f.**

What we hand on



Lecture 10 will be built to the identical shape as Lecture 8: *optimal control is the model-based origin of the control lineage; delete the dynamics f and you get policy-based RL.* The gain K that Act 3 solved for becomes a network $\mu_\theta(s)$ that is **trained to output what the Riccati equation used to compute**.

And the continuous $\arg\min$ of Act 1 — the wall Lecture 8 met — is answered there for good: stop searching for the minimiser and learn a function that emits it.

Lecture 7 solved the dynamic optimum with the tools of operations research. Lecture 9 solved *the same optimum* with the tools of control.

A value field (HJB), a closed form (LQR), and a trajectory law (Pontryagin) — three routes to one feedback rule $u = \gamma(t, x)$, best from wherever the system lands.

Questions?

Two traditions, one destination. Bellman wrote the same equation twice — once as a sum over a table, once as a partial differential equation — and Kalman found the single case where the second one can be solved. Everything after this is what happens when you are no longer told f.

APPENDIX

Backup slides

Complete arguments, kept out of the narrative.

Backup 1 — the HJB equation, derived in one step

Start from Bellman's optimality principle over a short interval δ:

$$V(t,x) = \min_{u \in U} \Big\{ g(t,x,u)\,\delta + V\big(t+\delta,\ x + f(t,x,u)\,\delta\big) \Big\}$$

Taylor-expand the second term about (t,x):

$$V\big(t+\delta,\ x+f\delta\big) = V(t,x) + \frac{\partial V}{\partial t}\,\delta + \frac{\partial V}{\partial x}\,f(t,x,u)\,\delta + o(\delta)$$

Substitute, cancel $V(t,x)$ from both sides — it is free of u, so it passes through the minimum — divide by δ and let $\delta \to 0$:

$$0 = \min_{u \in U} \Big\{ g(t,x,u) + \frac{\partial V}{\partial t} + \frac{\partial V}{\partial x} f(t,x,u) \Big\} \implies -\frac{\partial V}{\partial t} = \min_{u \in U} \Big\{ \frac{\partial V}{\partial x} f + g \Big\} \qquad \blacksquare$$

Compare Lecture 7's derivation of the Bellman optimality equation: identical, with $\delta \to 0$ in place of a one-step sum. The minimising u at each (t,x) is the optimal feedback — a static pointwise minimisation embedded inside a PDE for V.

Backup 2 — why HJB is sufficient: the verification argument

Suppose V is continuously differentiable and satisfies HJB. Take any admissible $\gamma \in \Gamma$, with trajectory x and terminal time T, alongside the candidate γ^* with x^* and T^*. Because the HJB right-hand side is a *minimum* over u, the arbitrary control can only do worse, while the minimising control attains it exactly:

$$g(t,x,u) + \partial_x V\, f(t,x,u) + \partial_t V \;\geq\; 0, \qquad\qquad g(t,x^*,u^*) + \partial_x V\, f(t,x^*,u^*) + \partial_t V \;\equiv\; 0$$

Along a trajectory the last two terms of each are exactly $\frac{d}{dt}V\big(t,x(t)\big)$, so integrating the first over $[0,T]$ and the second over $[0,T^*]$ gives

$$\int_0^{T} g\,dt + V\big(T,x(T)\big) - V(0,x_0) \;\geq\; 0, \qquad\qquad \int_0^{T^*} g^*\,dt + V\big(T^*,x^*(T^*)\big) - V(0,x_0) \;=\; 0$$

Eliminate $V(0,x_0)$ between them and apply the boundary condition $V(T,x) = q(T,x)$:

$$L(u) \;=\; \int_0^{T} g\,dt + q\big(T,x(T)\big) \;\geq\; \int_0^{T^*} g^*\,dt + q\big(T^*,x^*(T^*)\big) \;=\; L(u^*) \qquad \blacksquare$$

A solution of the PDE is therefore a **certificate**: no admissible strategy beats γ^*, from any initial state. This is exactly what Pontryagin's conditions do *not* give — they are stationarity conditions, satisfied by every extremal, optimal or not.

Backup 3 — LQR: the Riccati derivation, both times

Continuous time. $\dot{x} = Ax + Bu$, running cost $g = x^\top Q x + u^\top R u$. Guess $V(t,x) = x^\top P_t x$, so $\partial_x V = 2P_t x$ and $\partial_t V = x^\top \dot{P}_t x$. HJB reads

$$-x^\top \dot{P}_t x = \min_u \Big\{ 2x^\top P_t (Ax + Bu) + x^\top Q x + u^\top R u \Big\}$$

Minimise over u: $2B^\top P_t x + 2Ru = 0 \Rightarrow u^* = -R^{-1}B^\top P_t\, x$. Substitute back and match $x^\top(\cdot)x$ for all x:

$$-\dot{P}_t = A^\top P_t + P_t A - P_t B R^{-1} B^\top P_t + Q$$

Infinite horizon: $\dot{P} \to 0$, giving the algebraic Riccati equation $A^\top P + PA - PBR^{-1}B^\top P + Q = 0$ with a unique PSD solution, hence the constant gain $K = R^{-1}B^\top P$ and $u^* = -Kx$.

Discrete time. With $x_{t+1} = Ax_t + Bu_t$ and $V(z) = z^\top P z$, the backup is $z^\top P z = \min_w \{ z^\top Q z + w^\top R w + (Az + Bw)^\top P (Az + Bw) \}$, whose minimiser is $w^* = -(R + B^\top P B)^{-1} B^\top P A\, z$. Substituting and matching for all z:

$$P = Q + A^\top P A - A^\top P B \big(R + B^\top P B\big)^{-1} B^\top P A, \qquad K = \big(R + B^\top P B\big)^{-1} B^\top P A$$

The ARE has exactly one solution with $P = P^\top \succeq 0$, and it *is* the value function. Started from $P_1 = Q$, the recursion converges to it whenever (A, B) is controllable — value iteration, on a matrix. This discrete form is the one a learned critic implicitly targets in Lecture 10.

Backup 4 — Euler–Lagrange to the minimum principle

Adjoin the dynamics with a costate $\lambda(t)$ and form $H = g + \lambda^\top f$:

$$\bar{J} = \int_0^T \left[H(t, x, u, \lambda) - \lambda^\top \dot{x} \right] dt + q\big(x(T)\big)$$

Perturb the optimal control, $u^* \to u^* + \epsilon\, p(t)$, and require $\frac{d\bar{J}}{d\epsilon}\big|_{\epsilon=0} = 0$ for every admissible p. Integrating by parts and collecting terms gives

$$\dot{x}^* = \frac{\partial H}{\partial \lambda} = f, \qquad \dot{\lambda}^* = -\frac{\partial H}{\partial x}, \qquad \lambda(T) = \frac{\partial q}{\partial x}, \qquad \frac{\partial H}{\partial u} = 0$$

From Euler–Lagrange to Pontryagin. Stationarity, $\partial_u H = 0$, presumes u is interior to U. Pontryagin replaces it with the global condition $u^* = \arg\min_{u \in U} H(t, x^*, u, \lambda^*)$, which remains valid on the *boundary* of U — so it covers saturated actuators and bang-bang controls, where the derivative never vanishes.

A conserved quantity. Along a trajectory $\frac{dH}{dt} = \frac{\partial H}{\partial u}\dot{u} + \frac{\partial H}{\partial t}$; for a time-invariant problem the second term vanishes and the first vanishes precisely when u minimises H at every instant. So H is constant along an extremal and drifts along anything else — the invariant the Act 4 widget plots. The minimum principle is also the standard way to *state* equilibrium conditions for dynamic games, which is how Lecture 13 uses it.